

TypedQL: um SGBD relacional cujo esquema vive no nível de tipos

Vitor Martins Bueno Lima | Desenvolvimento Orientado a Tipos | UFABC | 2026 | Video: <https://youtu.be/kNeXjRnFuDk>

Pergunta

Em um banco de dados tradicional quase todo erro é de runtime: coluna inexistente, tipo incompatível, NULL inesperado, plano físico malformado. **Quantos desses o sistema de tipos do Haskell elimina antes de rodar, e a que custo?** TypedQL e a resposta empírica: 2.084 linhas, GHC 9.6.7, sem dependência além de `base`, `containers` e `text`.

Construção

O esquema de uma tabela é um tipo: `Vendors = '[ "vendor_code" := TText, "cnpj" :? TText ]`. Toda a pilha é indexada por ele.

Camada	Módulo	Ideia
Dados	<code>Schema</code> , <code>Row</code>	lista heterogênea indexada; acesso por <i>prova</i> <code>KnownIndex</code>
Consulta	<code>Expr</code> , <code>Algebra</code>	nulabilidade calculada por type family; estágio <code>Logical/Physical</code> no tipo
Superfície	<code>Frontend.Static</code> , <code>.Dynamic</code>	quasiquote de SQL em compile time; existencial <code>SomeTable</code> + singleton
Plano	<code>Optimize</code> , <code>Engine</code>	catamorfismo indexado de reescrita; operadores físicos; EXPLAIN e custo

Um único combinador, `hcata :: (forall x. QueryF g x -> g x) -> QueryNode s -> g s`, serve para otimizar, contar nós e escolher algoritmo físico. A álgebra do plano foi escrita uma vez.

Resultado: 14 erros que deixaram de existir

`scripts/testar_negativos.py` compila cada arquivo de `negativos/` e falha se algum compilar, conferindo também a mensagem do GHC. Estado: **14 de 14 rejeitados**, ao lado de 90 testes de valor passando e zero aviso sob `-Wall -Wextra`. Sete dos quatorze produzem mensagem escrita por mim via `TypeError`, não ruído do compilador:

```
TypedQL: este filtro pode ser NULL, então ele não decide nada. Um WHERE precisa escolher entre verdadeiro e falso. Trate o NULL antes.
```

Os dois achados

1. Uma reescrita de otimizador errada em SQL virou erro de tipo. Absorver o `WHERE` de cima na condição `ON` é válido em `INNER JOIN` e **incorreto** em `LEFT JOIN`, porque muda quais linhas recebem preenchimento nulo. Não foi preciso proibir: o predicado do `LJoin` tem tipo `Predicate (Append 1 r)` e o filtro de cima tem `Predicate (Append 1 (MakeNullable r))`. Os dois não unificam. A regra de semântica relacional caiu fora do espaço de programas válidos sem que eu escrevesse uma verificação.

2. Hash join com chave nulável não é construível. Em SQL `NULL = NULL` não é verdadeiro. Um hash join ingenuo joga todos os `NULL` no mesmo balde e casa todos com todos, devolvendo linhas que não existem no resultado correto: um bug silencioso e clássico. O construtor `HashJoin` exige `ColumnOf k1 l ~ Col k1 t NotNull` nos **dois** lados, com o mesmo `t`. O operador errado não tem representação.

Custo, medido e não estimado

Com 50 linhas de cada lado, o laço aninhado faz 2.500 comparações e o hash join faz 100, e os dois devolvem as mesmas linhas na mesma ordem. O custo de *programar* foi outro. Três registros honestos:

- Type family que não reduz.** Uma família só avalia se alguém consome o resultado. `Proxy` não dispara `TypeError`; foi preciso forçar com uma constraint real (`All Show`).
- Append não é injetiva.** Dois negativos de hash join passavam pelo motivo errado (ambiguidade, não incompatibilidade) até eu anotar `scan "a" [] :: PhysOp A`. Testar um teste negativo também é necessário.
- Limite que ficou.** Promover `ON (a = b)` a hash join automaticamente não typecheca: a prova que `ECol` carrega fala de `Append 1 r`, e `HashJoin` quer uma prova em cada lado. `Append` apagou de qual lado a coluna veio. A correção seria um `Predicate2 1 r`, reescrita do módulo de expressões.

Conclusão

A tese se sustenta com uma ressalva. Erros de nome, de tipo, de nulabilidade, de ordem de estágio e de preservação de esquema foram eliminados por construção. Mas o tipo não captura ordem de linhas: o `flip` em `M.fromListWith (flip (++))` e a diferença entre a saída do hash join casar ou não com a do laço aninhado, e nenhum tipo reclamaria. **O sistema de tipos elimina a classe estrutural de erros; a observacional continua sendo trabalho de teste.** Reproduzir: `stack test` e `python scripts/testar_negativos.py`. Video: <https://youtu.be/kNeXjRnFuDk>